

Summer School · EIV 2026

# GNNs & LLMs: Reasoning Beyond Text

**Dr. Daniel F. Pérez-Ramírez**

Guest at Escuela Colombiana de Ingeniería Julio Garavito  
2026-07-02

5 days · 24 hrs · In-Person



---

SECTION 0

# Module II Recap

---

## Module 2 Recap

**P1. ¿Cuál de estas NO es una fase de entrenamiento de un LLM? (una opción correcta)**

- a.** Pre-training (predicción del siguiente token sobre grandes corpus de texto)
- b.** Supervised fine-tuning (SFT / instruction tuning)
- c.** Alignment con RLHF (Reinforcement Learning from Human Feedback)
- d.** Retrieval-Augmented Generation (RAG)

---

## Module 2 Recap

### 2. ¿Qué dicen las scaling laws de los LLMs descubiertas en 2020? (una opción correcta)

- a. El loss mejora de forma continua, como una ley de potencias, al aumentar tamaño del modelo, datos y cómputo
- b. Existe un tamaño de modelo óptimo fijo, y superarlo degrada el desempeño
- c. El desempeño mejora solo aumentando los datos, manteniendo fijo el tamaño del modelo
- d. Duplicar los parámetros duplica el desempeño en una relación lineal

---

## Module 2 Recap

### 3. ¿Qué son los vectores embedding? (una opción correcta)

- a. Vectores one-hot que asignan un índice único a cada palabra del vocabulario
- b. Vectores densos que ubican tokens/documentos con significado similar en posiciones cercanas de un espacio continuo
- c. La matriz de attention que relaciona cada token con todos los demás
- d. Los logits de salida del modelo justo antes del softmax

---

## Module 2 Recap

### 4. ¿Cuál es el propósito de attention? (una opción correcta)

- a.** Conectar cada token únicamente con el token inmediatamente anterior
- b.** Comprimir toda la secuencia en un único vector de tamaño fijo antes de decodificar
- c.** Asignar a cada posición un peso fijo aprendido, independiente del contenido
- d.** Ponderar dinámicamente la relevancia de otros tokens según el contexto para representar cada token

---

## Module 2 Recap

**5. El modelo original del Transformer (Vaswani et al., 2017) se compone de un ENCODER y un DECODER. (verdadero / falso)**

- a.** Verdadero
- b.** Falso

---

## Module 2 Recap

**6. La mayoría de los LLMs tipo chatbot actuales son solo DECODER (decoder-only).  
(verdadero / falso)**

- a.** Verdadero
- b.** Falso

---

## Module 2 Recap

### 7. El propósito del positional encoding es... (una opción correcta)

- a. Codificar la importancia semántica de cada token
- b. Reducir el costo computacional de la attention
- c. Inyectar información del orden de los tokens, que la self-attention por sí sola no captura (es permutation-invariant)
- d. Indicar qué tokens deben enmascarse durante el entrenamiento

---

## Module 2 Recap

### 8. ¿Cuál es la motivación de usar varias cabezas/heads de atención (multi-head attention)? (una opción correcta)

- a. Atender en paralelo a distintos subespacios o tipos de relación (sintácticas, semánticas, etc.)
- b. Aumentar la profundidad (número de capas) del modelo
- c. Que todas las heads produzcan el mismo resultado y así ganar robustez
- d. Sustituir a la capa feed-forward posterior a la attention

---

## Module 2 Recap

### 9. ¿Cuál es la función del tokenizer? (una opción correcta)

- a. Convertir los tokens en vectores embedding densos
- b. Segmentar el texto en subunidades (p. ej. subwords) mapeadas a IDs enteros, y viceversa
- c. Asignar a cada token su probabilidad de aparición
- d. Eliminar stopwords y normalizar el texto antes del entrenamiento

## Module 2 Recap

### 10. ¿Cuál es la diferencia entre greedy decoding y top-k / top-p (nucleus) decoding? (una opción correcta)

- a. Greedy evalúa toda la secuencia futura; top-k/top-p solo el siguiente token
- b. Top-k y top-p son idénticos: ambos fijan un número constante de candidatos
- c. Greedy muestrea al azar, mientras que top-k/top-p siempre toman el token más probable
- d. Greedy toma el argmax en cada paso; top-k restringe a los k tokens más probables y top-p al conjunto mínimo cuya probabilidad acumulada supera p, muestreando dentro de ese conjunto

---

## Module 2 Recap

### 11. ¿Cuál es la diferencia entre un Transformer denso y un modelo Mixture-of-Experts (MoE)? (una opción correcta)

- a. En un MoE cada experto procesa toda la secuencia y luego se promedian sus salidas
- b. En un MoE, un router activa solo algunos "expertos" por token (activación dispersa): muchos parámetros totales, poco cómputo por token
- c. Un MoE reemplaza la self-attention por varias redes expertas
- d. Un MoE entrena un modelo distinto por cada tipo de tarea y elige uno en inferencia

## Module 2 Recap

### 12. ¿Cuáles son las fases de un sistema RAG? (una opción correcta)

- a.** Retrieval de documentos, fine-tuning del LLM sobre esos documentos y Generation
- b.** Embedding de la pregunta, re-entrenamiento del modelo y decoding
- c.** Indexing (chunking + embedding + almacenamiento), Retrieval (recuperación por similitud) y Generation (respuesta aumentada con el contexto recuperado)
- d.** Tokenization, indexing y attention

---

## Module 2 Recap

### 13. ¿Qué factor limita el tamaño máximo de los chunks al construir la VectorDB en RAG? (una opción correcta)

- a. Retrieval de documentos, fine-tuning del LLM sobre esos documentos y Generation
- b. Embedding de la pregunta, re-entrenamiento del modelo y decoding
- c. Indexing (chunking + embedding + almacenamiento), Retrieval (recuperación por similitud) y Generation (respuesta aumentada con el contexto recuperado)
- d. Tokenization, indexing y attention

---

## Module 2 Recap

**14. Para evaluar qué tan bien el componente de retrieval de tu RAG recupera los chunks relevantes, ¿cuál es la métrica más apropiada? (una opción correcta)**

- a.** El context window (longitud máxima de entrada) del embedding model: un chunk más largo se trunca y pierde información
- b.** El context window del LLM generador que produce la respuesta final
- c.** El número de dimensiones del vector embedding
- d.** La temperature usada durante la generación

## Module 2 Extension --- Embedding vectors

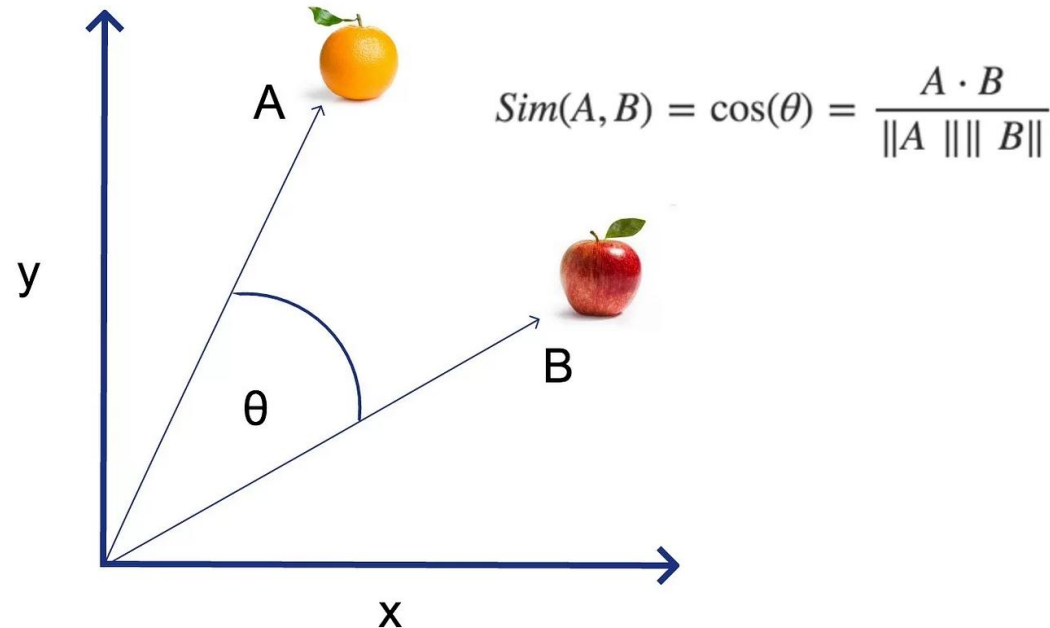
|        |                      |                  |                                                                |
|--------|----------------------|------------------|----------------------------------------------------------------|
| Perro  | [ 0 , 1, 0, ..., 0 ] | One-hot encoding | Not an embedding!<br>Orthogonal (in general)!<br>Non-learnable |
| Gato   | [ 0 , 1, 0, ..., 0 ] |                  |                                                                |
| Bogota | [ 0 , 0, 0, ..., 1 ] |                  |                                                                |

Perro [ 0.01 , 1.23, 0.34, ..., 1.43 ]

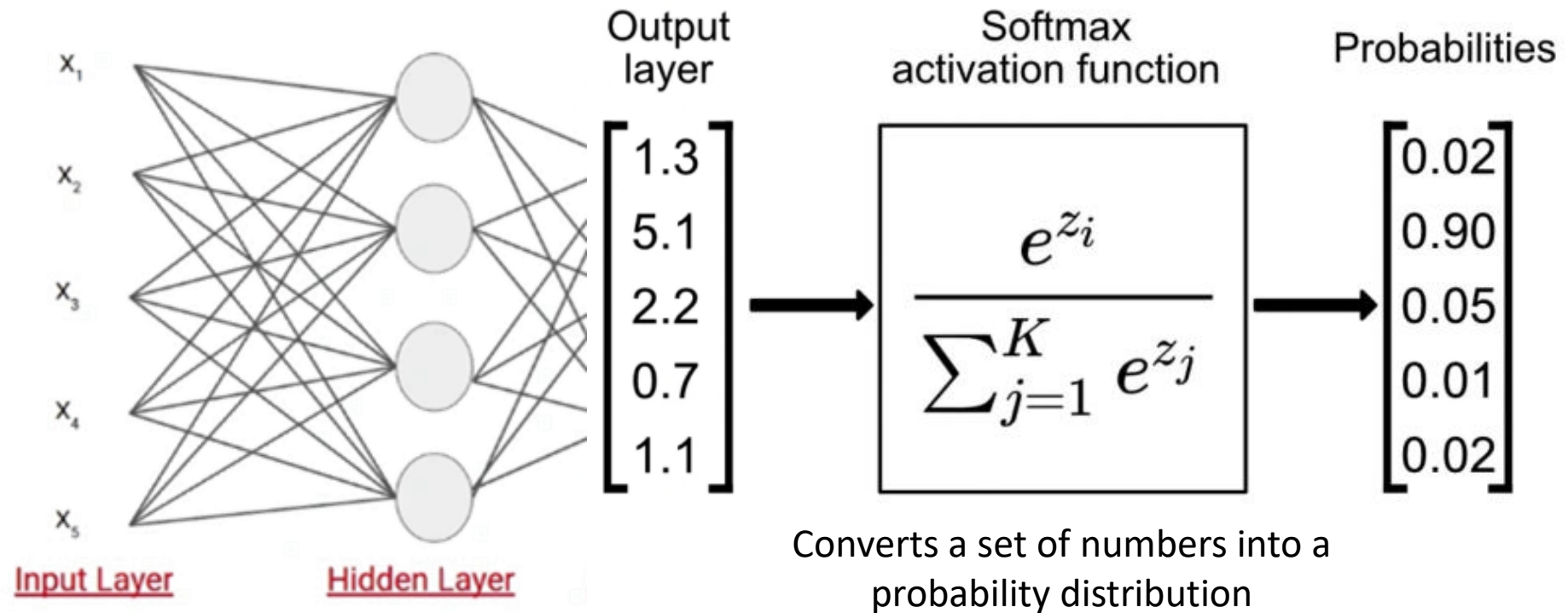
Embedding

Real-valued and dense vector.  
Learnt!

Similarity: COSINE



## Module 2 Extension --- Temperature

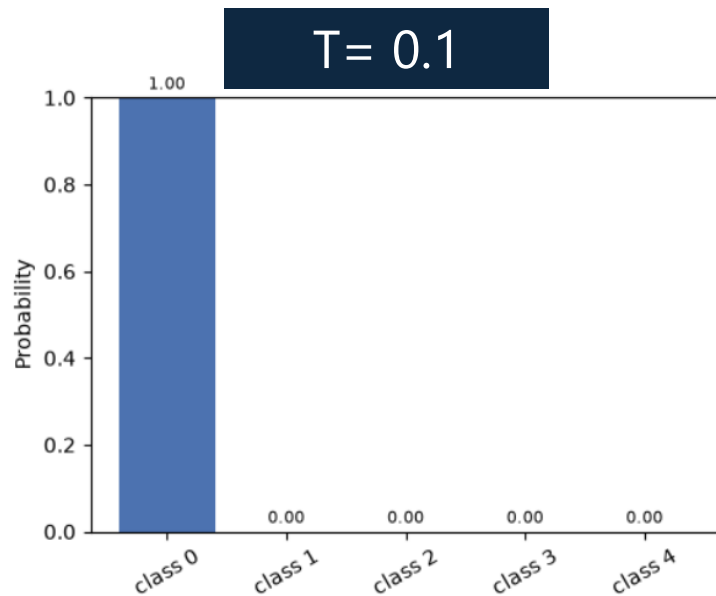


**NO temperature**

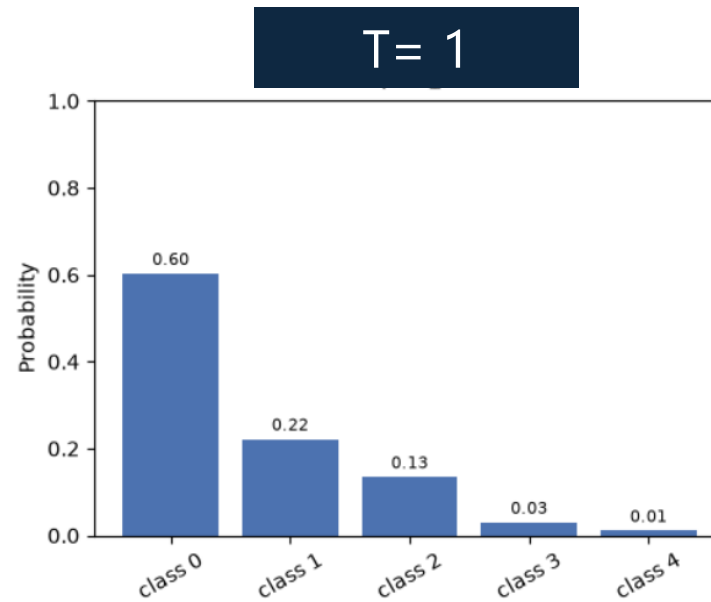
## Module 2 Extension --- Temperature

**WITH temperature**

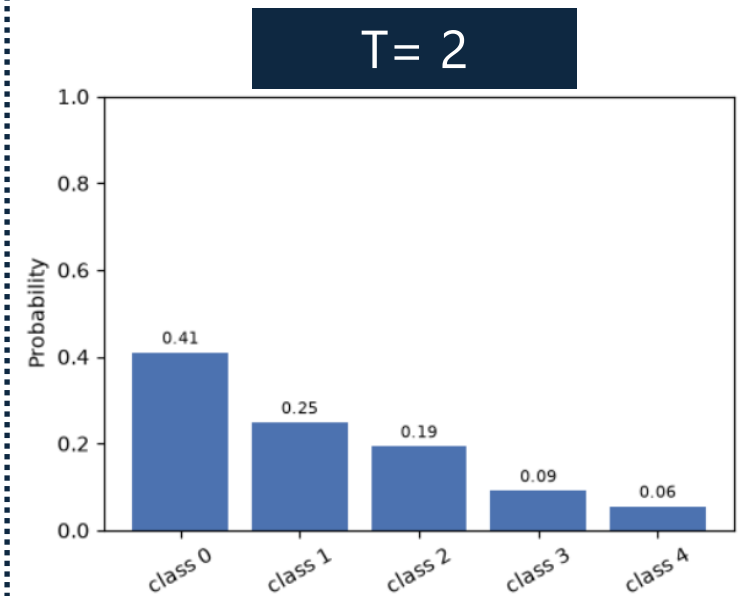
$$\text{softmax}(z_i; T) = \frac{\exp(z_i/T)}{\sum_{j=1}^N \exp(z_j/T)}$$



“sharper” distribution  
→ Deterministic



Standard softmax



“flatter” distribution  
→ Increase randomness

# Course Overview

I · 5-6 hrs

## **ML Foundations**

ML best practices. Train first ML model.

II · 5-6 hrs

## **LLMs + RAG**

Fundamentals of LLMs and first RAG system.

III · 5-6 hrs

## **GNNs**

Encode relational structure.

IV · 5-6 hrs

## **LLMs + GNNs**

Combine all three end-to-end

V · 3-4 hrs

## **Wrap-up + Present**

Demo your course project.

# From Module II to Module III

## WHERE RAG STOPPED

- RAG can ground LLMs in documents
- But it cannot represent how entities **relate**
- Multi-hop questions break — text chunks have **no edges**

## WHERE WE GO TODAY

**Graphs are the natural language of relational data — and GNNs learn on them.**

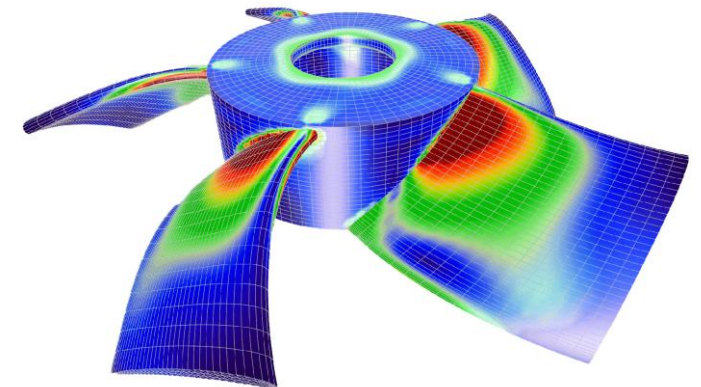
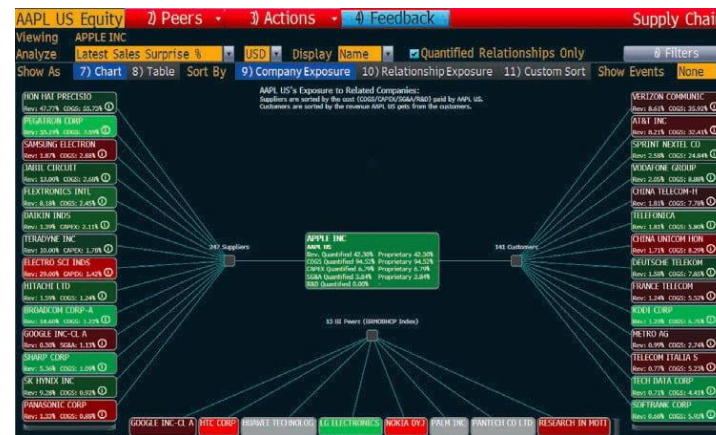
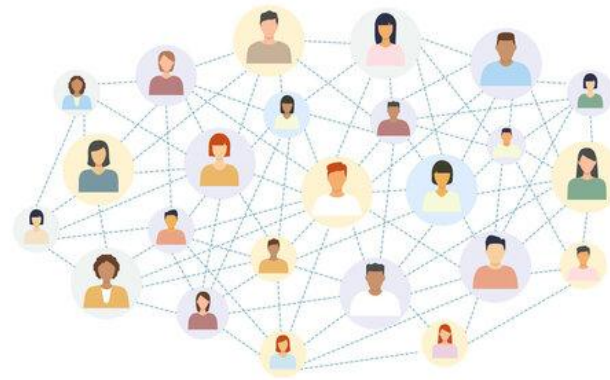
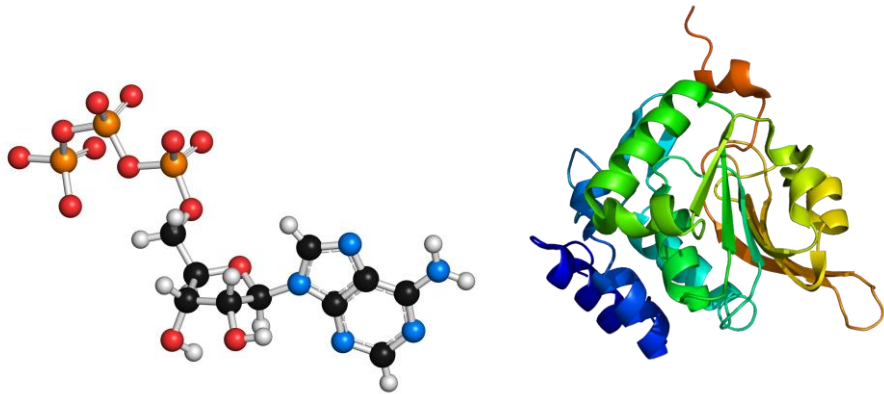
**If your data has entities and relationships, you probably have a graph — and a GNN is the right model.**

---

SECTION 1

# Graph-Structured Data

# Graphs Are Everywhere



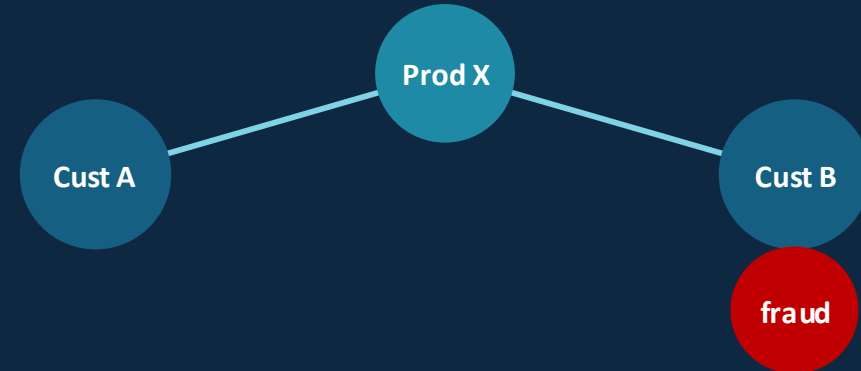
# Tabular vs Graph Data

TABLE — ROWS ARE INDEPENDENT

| customer | bought    |
|----------|-----------|
| A        | product X |
| B        | product X |
| C        | product Y |

The shared product is **obscure** — two rows that look independent.

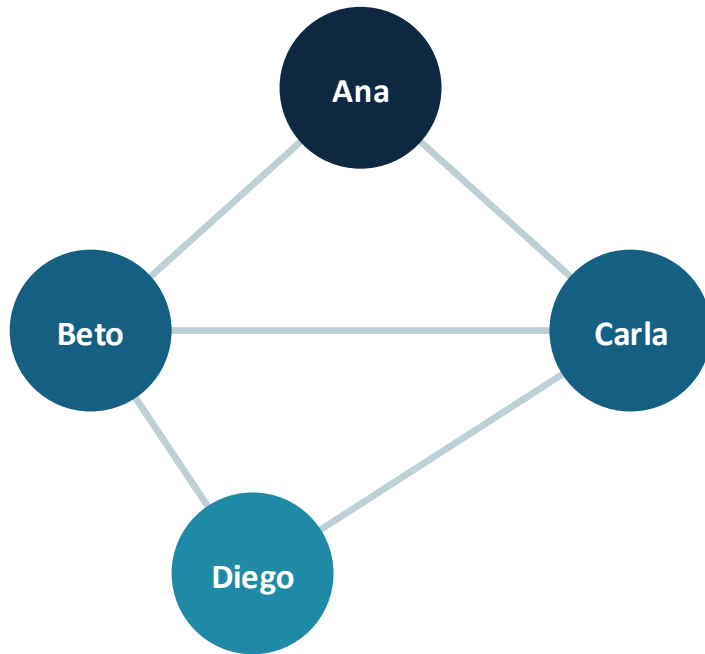
GRAPH — STRUCTURE IS SIGNAL



Cust B is one hop from a **known fraudster**. The graph knows. The table doesn't.

**Structure is signal. Tables lose it; graphs preserve it.**

# What is a Graph?



**Nodes (N)** entities — people, papers, products, atoms

**Edges (E)** relationships — friendship, citation, purchase

**Graph (G)**  $G = \langle N, E \rangle$

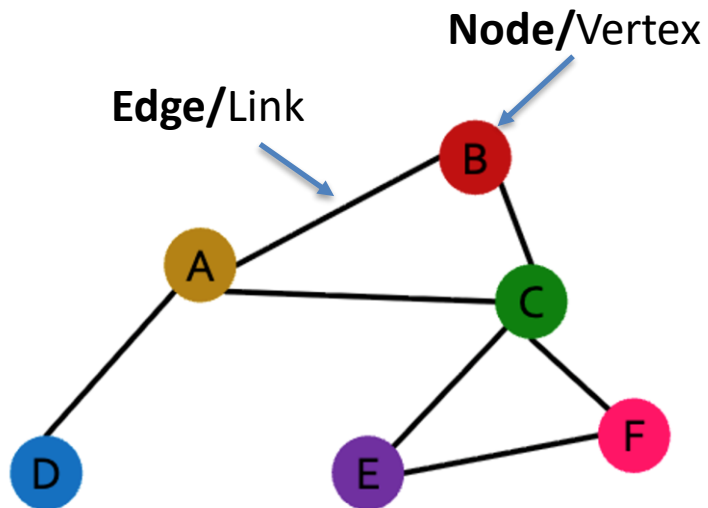
**Directed** vs. undirected — do edges have a direction?

**Weighted** vs. unweighted — do edges carry a strength?

**Homo-** vs. heterogeneous — one type of node, or many?

# Basic Concepts

Undirected Graph



Adjacency Matrix

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A |   |   |   |   |   |   |
| B |   |   |   |   |   |   |
| C |   |   |   |   |   |   |
| D |   |   |   |   |   |   |
| E |   |   |   |   |   |   |
| F |   |   |   |   |   |   |

Node Feature Matrix

|   | 1 | 2 |
|---|---|---|
| A |   |   |
| B |   |   |
| C |   |   |
| D |   |   |
| E |   |   |
| F |   |   |

Each row is a vector of node features

Node Degree Vector

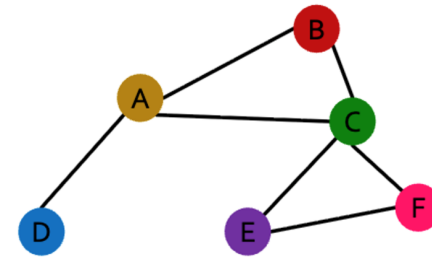
| A | 3 |
|---|---|
| B | 2 |
| C | 4 |
| D | 1 |
| E | 2 |
| F | 2 |

Can be expressed as a diagonal matrix as well

# Node Feature Matrix

Describes the features of each node in the graph.  
Dimension  $N \times D$ , with  $D$  features per node.

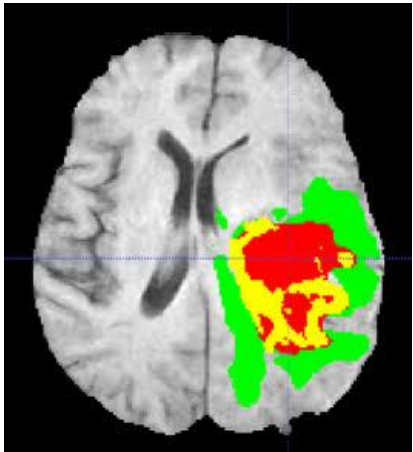
Undirected Graph



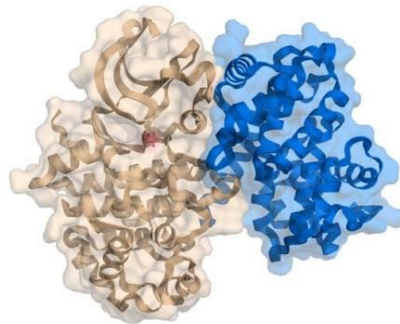
Input Node Feature Matrix

|   |  |  |
|---|--|--|
| A |  |  |
| B |  |  |
| C |  |  |
| D |  |  |
| E |  |  |
| F |  |  |

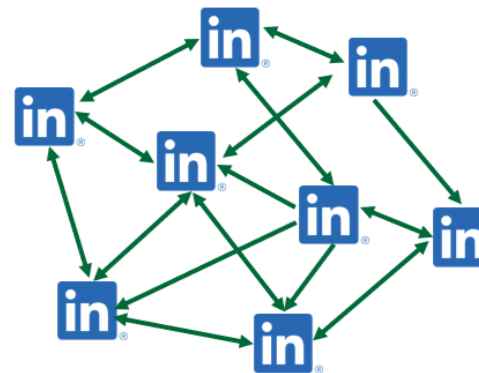
CT image



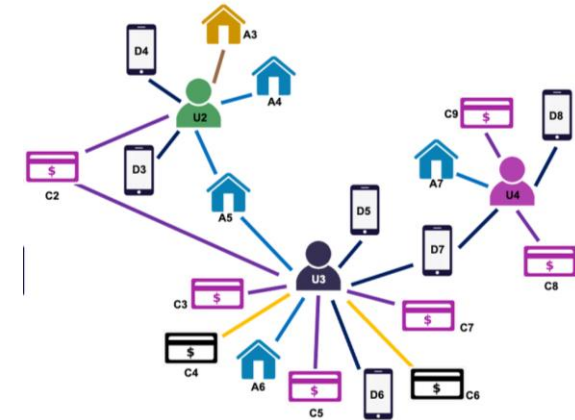
Protein-Protein Interaction



LinkedIn Network

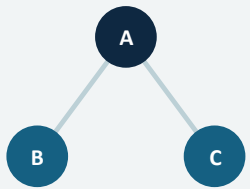


E-commerce Product-Customer catalog



# Edge Type Change the Adjacency Matrix

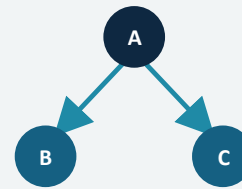
Undirected



|   | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 1 | 0 | 0 |
| C | 1 | 0 | 0 |

Symmetric:  $A_{ij} = A_{ji}$

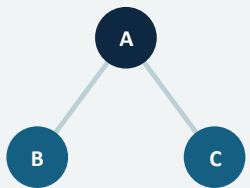
Directed



|   | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 0 | 0 | 0 |
| C | 0 | 0 | 0 |

Asymmetric:  $A_{ij} \neq A_{ji}$

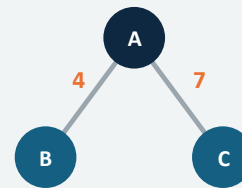
Unweighted



|   | A | B | C |
|---|---|---|---|
| A | 0 | 1 | 1 |
| B | 1 | 0 | 0 |
| C | 1 | 0 | 0 |

Entries are 0 / 1 — edge or no edge

Weighted  $A-B = 4, A-C = 7$



|   | A | B | C |
|---|---|---|---|
| A | 0 | 4 | 7 |
| B | 4 | 0 | 0 |
| C | 7 | 0 | 0 |

Entry = edge weight, not just 1

# Representing Graphs in Code

## Adjacency matrix

```
A B C
A [ 0 1 1 ]
B [ 1 0 1 ]
C [ 1 1 0 ]
```

**$O(N^2)$  memory — impractical for large graphs.**

## Edge list ← practical

```
edge_index = [
    [0, 1, 2], # source indices
    [1, 2, 0]  # target indices
]
```

**Stores only existing edges — memory-efficient.**

## PyG Data object

```
data.x          # N × F
data.edge_index  # 2 × E
data.y          # labels
```

Node features + edges + labels — used in the lab.

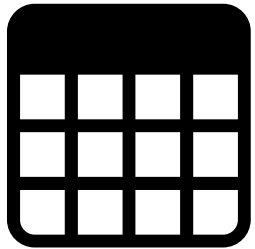
---

SECTION 2

# Machine Learning for Graphs

# ML for Graphs

## Tabular Data



Fully-connected Neural Network  
(Multi-layer Perceptron)



## Text/Speech

This is a sample text



Tokenization

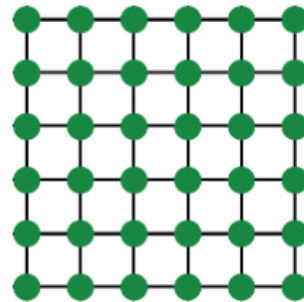


This is a sample text

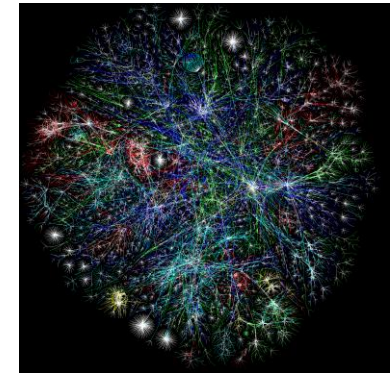
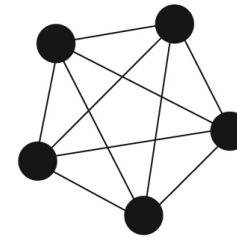
## Images



Convolutional  
Neural Network



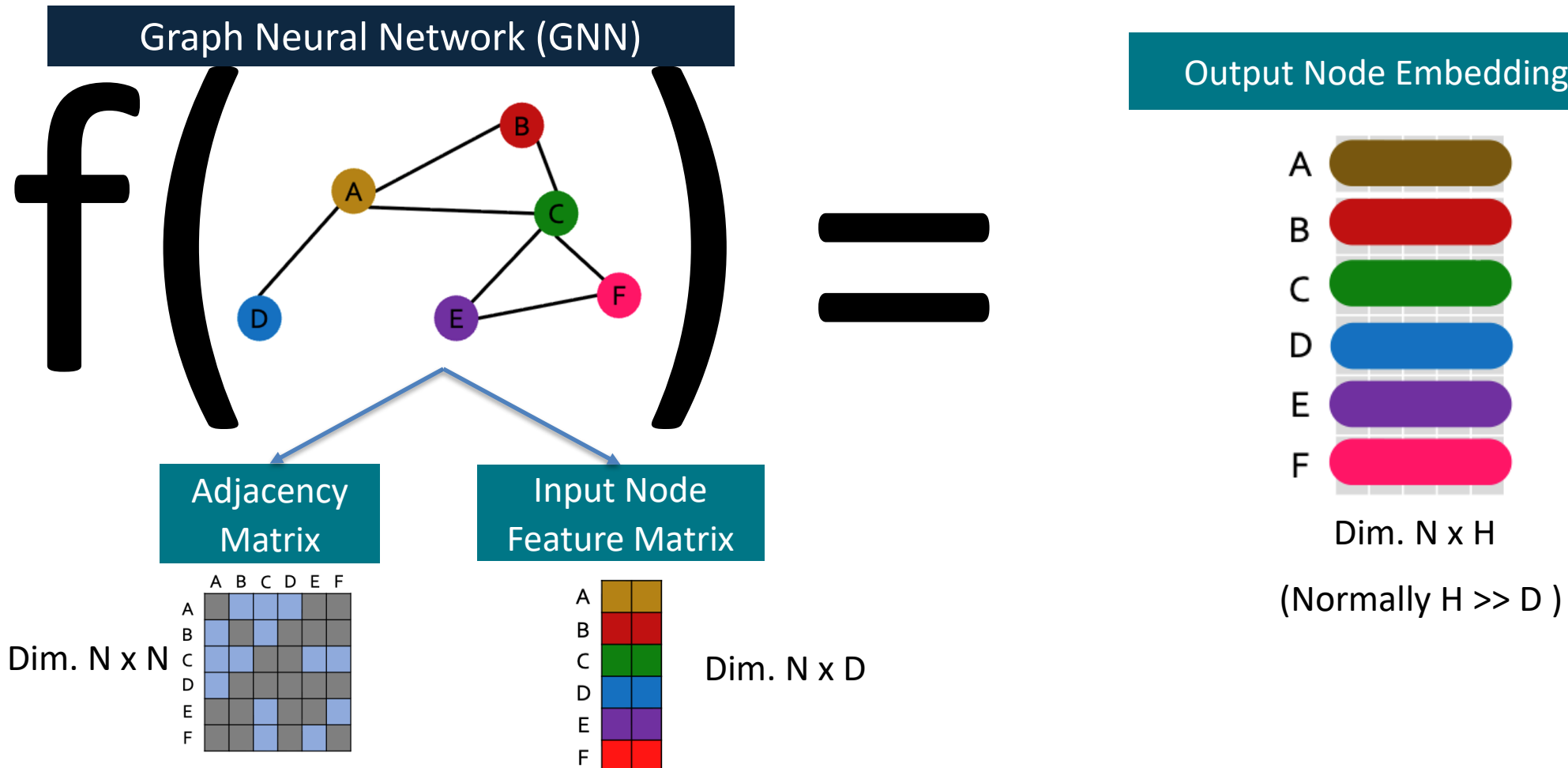
## Graphs



Conventional deep  
learning unsuitable for  
processing graphs!

Fixed input and output structures

# Graph Representation Learning



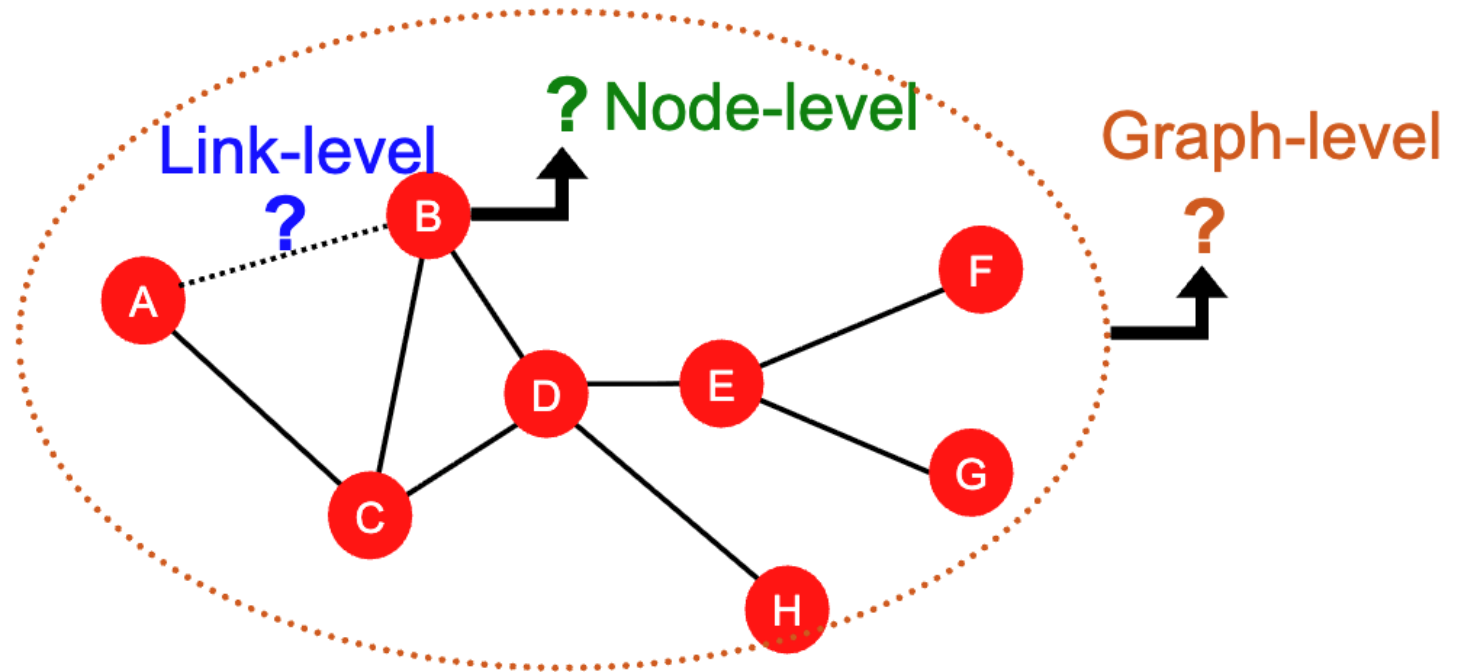
# Graph Representation Learning

- Node-level prediction
- Link-level prediction
- Graph-level prediction

## Intuition

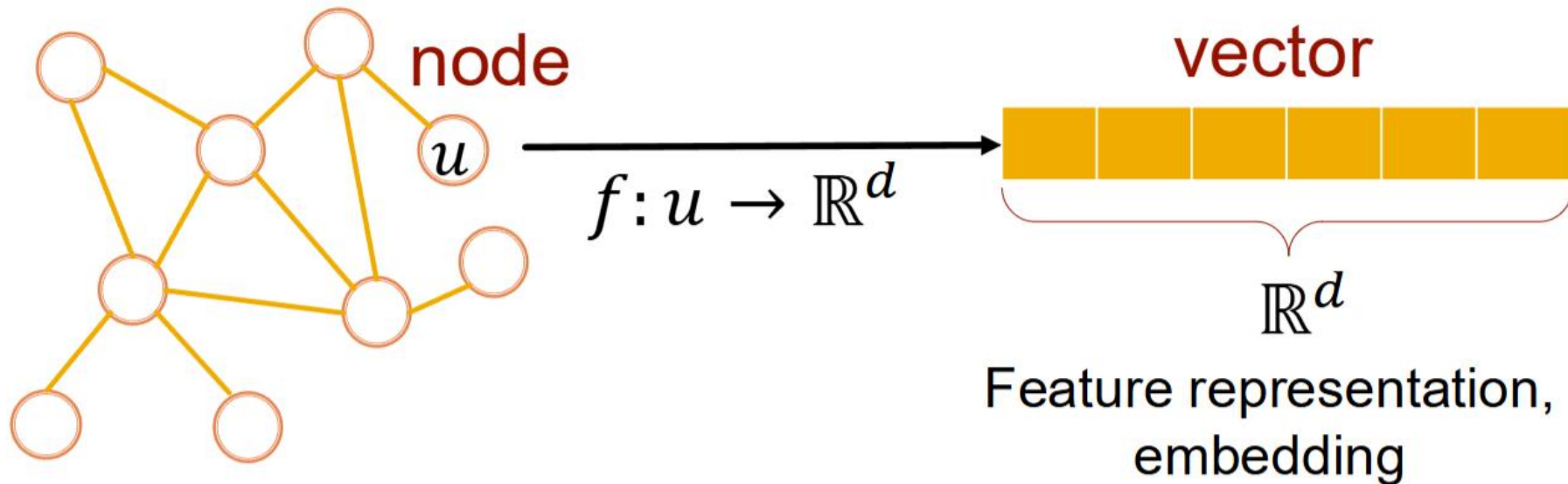
1. Compute embedding vectors for each node **considering** the local node neighborhood

2. Use the node embeddings to perform the desired task



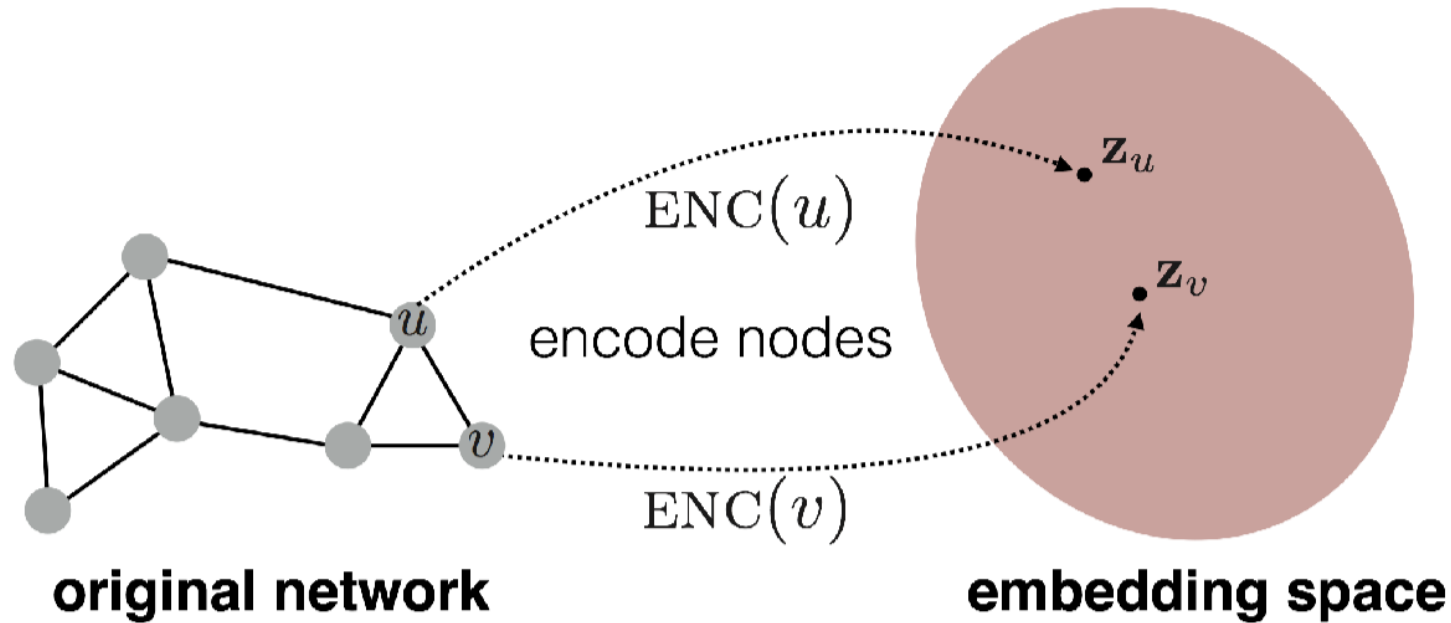
# Node Embeddings

Efficient task-independent feature learning for machine learning with graphs!



# Node Embeddings

**Goal:** similarity in the embedding space (e.g., dot product) approximates similarity in the graph



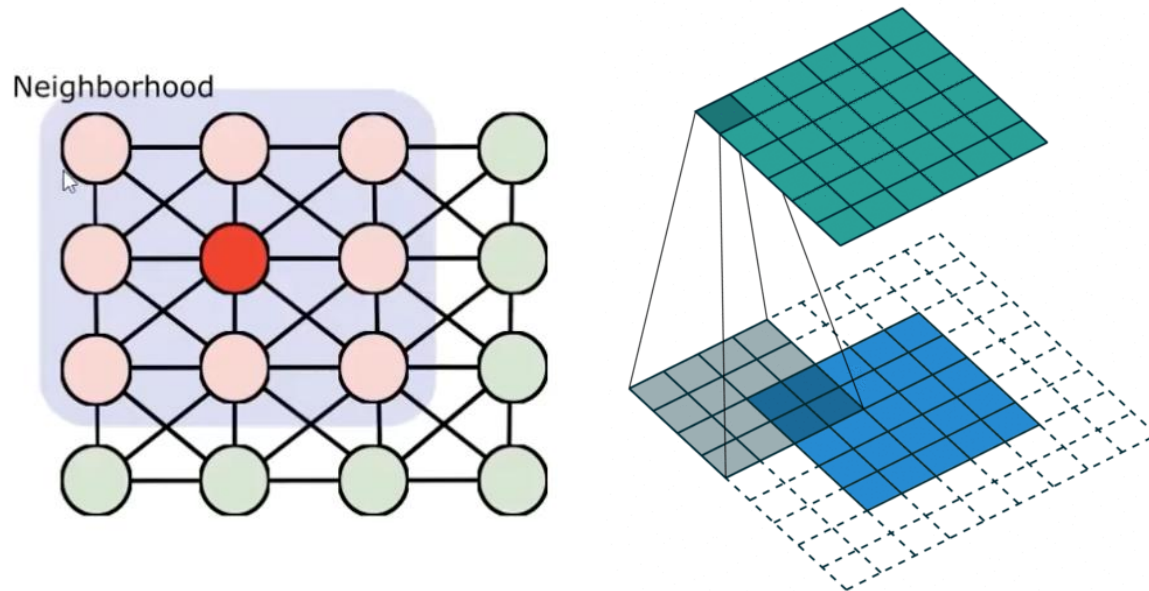
Goal:  $\text{similarity}(u, v)$   $\approx \mathbf{z}_v^T \mathbf{z}_u$

in the original network

Similarity of the embedding

# CNNs vs GNNs

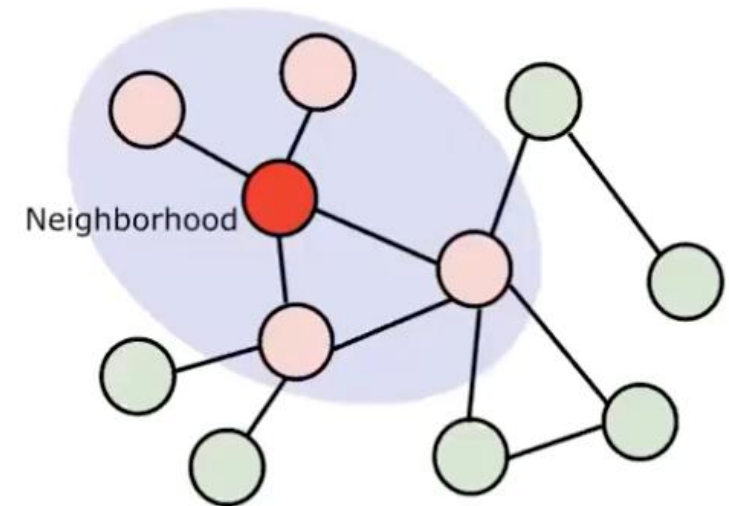
Convolutional Neural Network  
(CNN)



There is no fixed notion of locality or sliding window for a graph!



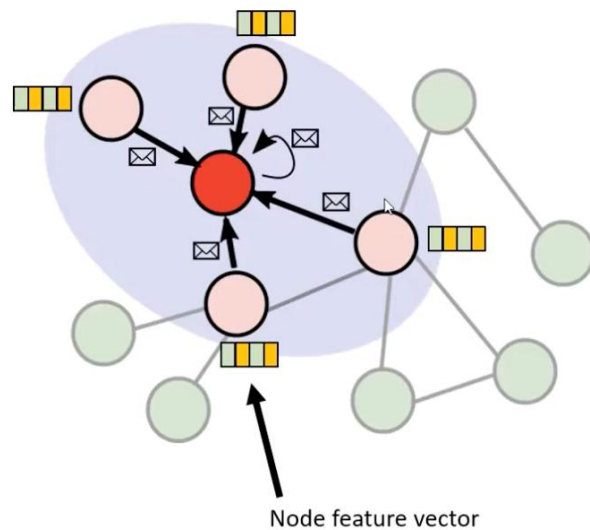
Graph Neural Network (GNN)



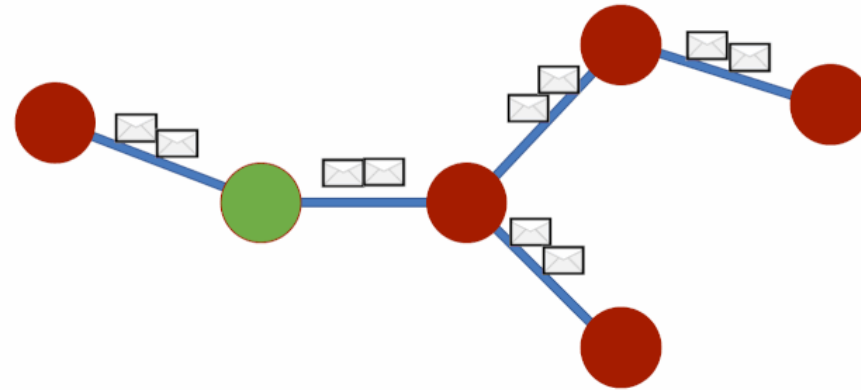
Message-passing operations

# Message Passing

Node-level



Graph-level



The message passing operations build our output embedding matrix

<https://www.outsystems.com/blog/posts/graph-neural-networks/>

# Message Passing

**Message** from node  $j \rightarrow i$ :

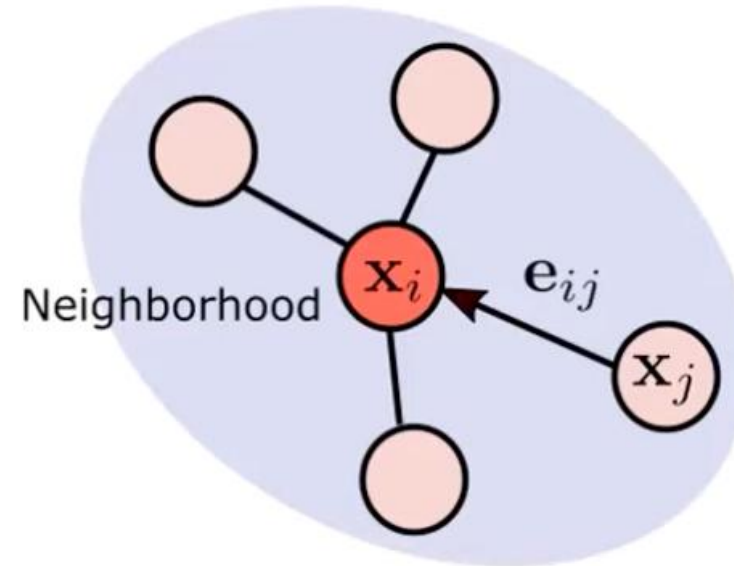
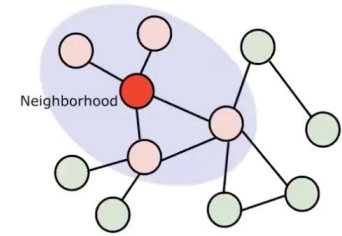
$$m_j = m(x_i, x_j, e_{ij})$$

**Aggregate** messages from all neighbors:

$$a = a(m_j | j \text{ neighborhood of } i)$$

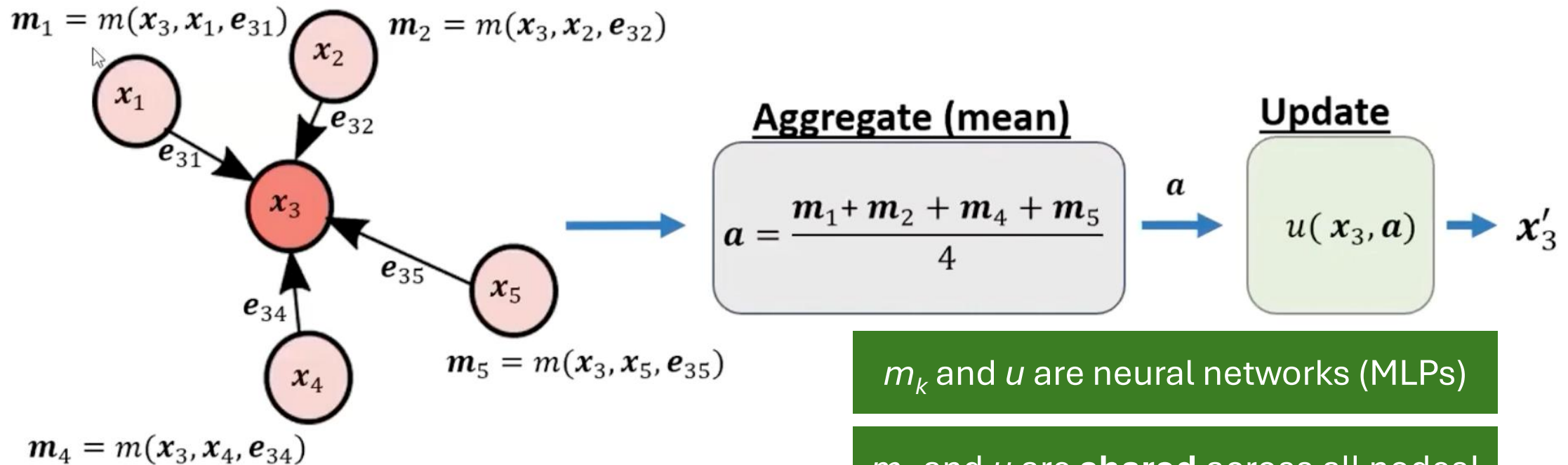
**Update** node  $i$ :

$$x'_i = u(x_i, a)$$



# Message Passing — A one-layer GNN

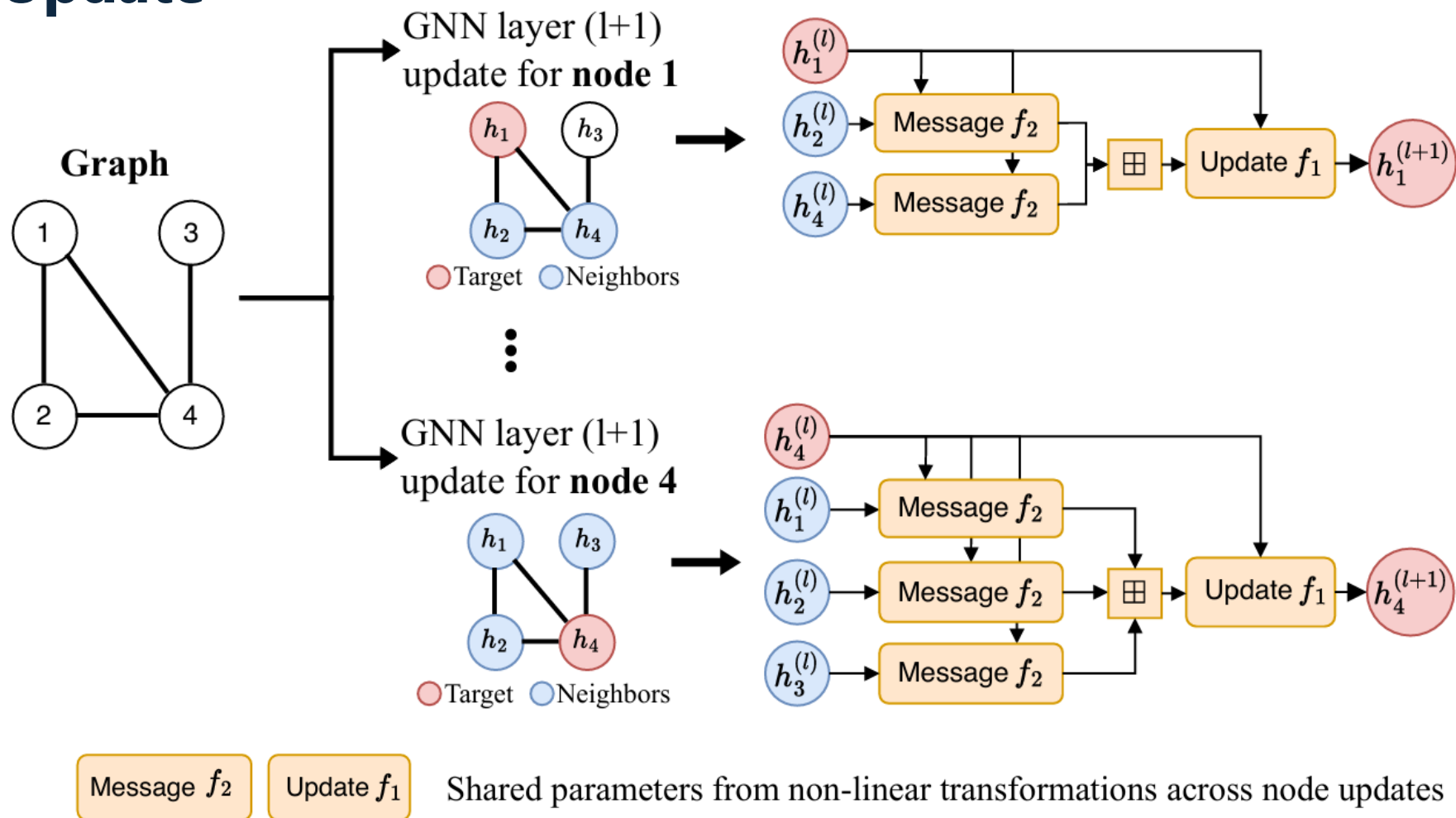
## Message



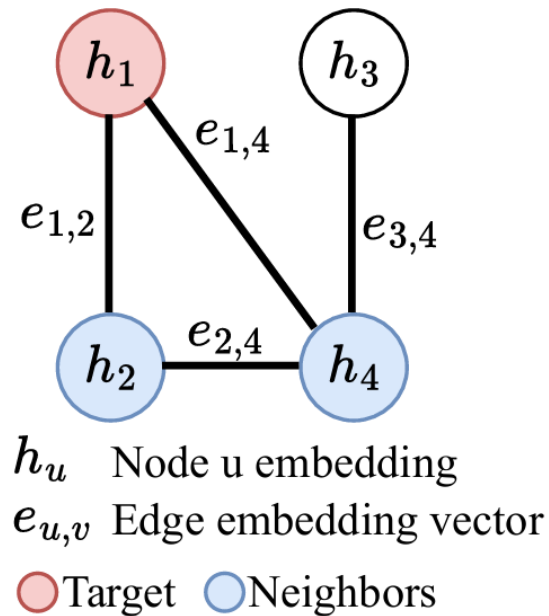
$m_k$  and  $u$  are neural networks (MLPs)

$m_k$  and  $u$  are **shared** across all nodes!

# GNN Update



# Detailed GNN Update for Node 1



**1. Message  $f_2$**   $a_{1,2}^{(l)} = f_2(h_1^{(l)}, h_2^{(l)}, e_{1,2})$   
 $a_{1,4}^{(l)} = f_2(h_1^{(l)}, h_4^{(l)}, e_{1,4})$

Non-linear transformation from neighboring nodes (e.g., MLP or FCNN)

**2. Aggregate  $\boxplus$**

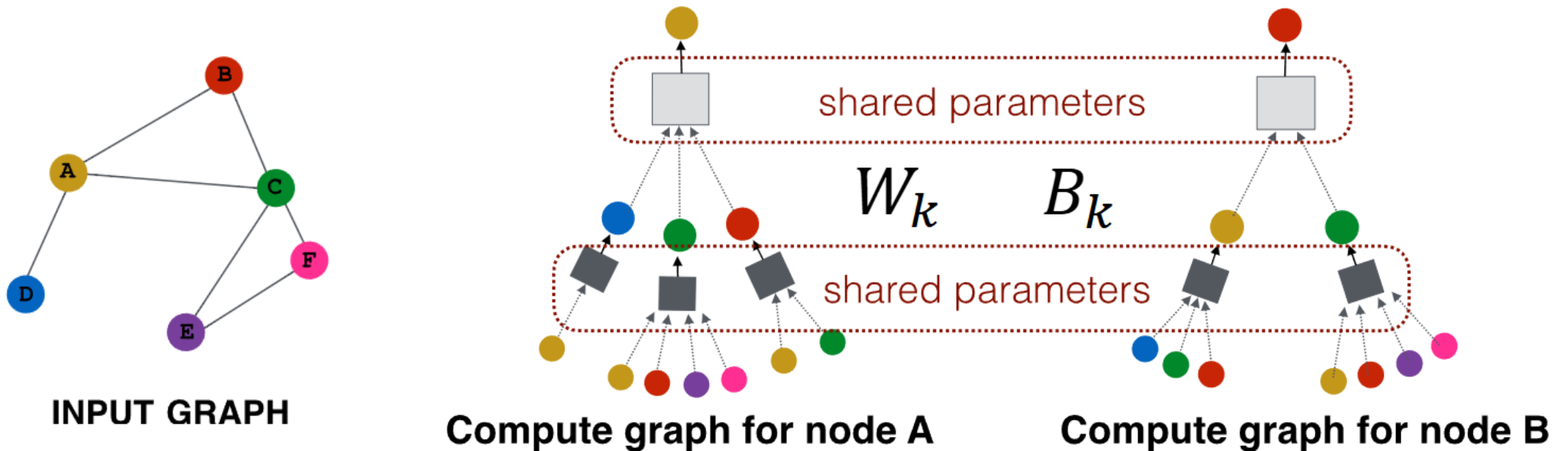
$$a_{tot}^{(l)} = \sum_{u \in \mathcal{N}(1)} f_2(h_1^{(l)}, h_u^{(l)}, e_{1,u}) = a_{1,2}^{(l)} + a_{1,4}^{(l)}$$

Commutative aggregation operation of the messages (e.g., sum)

**3. Update  $f_1$**   $h_1^{(l+1)} = f_1(h_1^{(l)}, a_{tot}^{(l)})$

Non-linear transformation between aggregation and previous node embeddings

## Another Perspective: Computation Graphs per Node



# GNNs are Permutation Equivariant

In other words: reordering the inputs reorders the output the same way.

For a Graph  $G = \langle A, X \rangle$  the function  $f$  is **permutation equivariant** if:

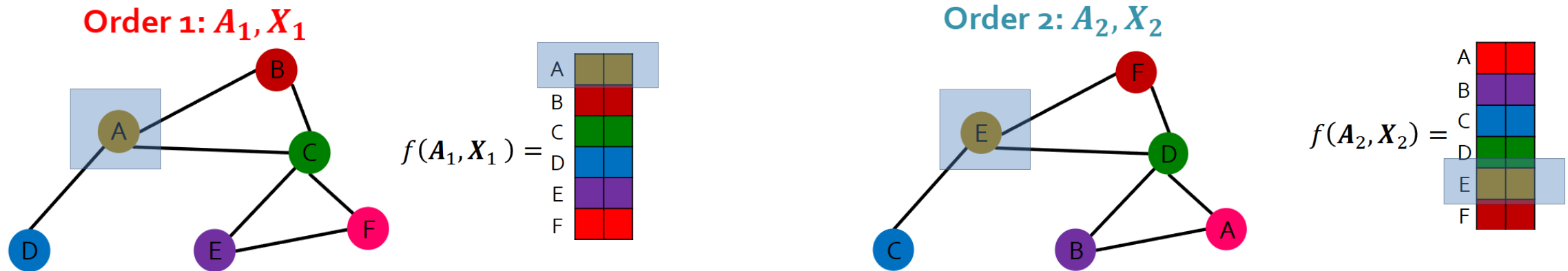
$$f(A_i, X_i) = f(A_j, X_j) \quad \text{For any ordering } i, j$$

$V$  is the **vertex set**

$A$  is the **adjacency matrix** (assume binary)

$X \in \mathbb{R}^{|V| \times m}$  is a matrix of **node features**

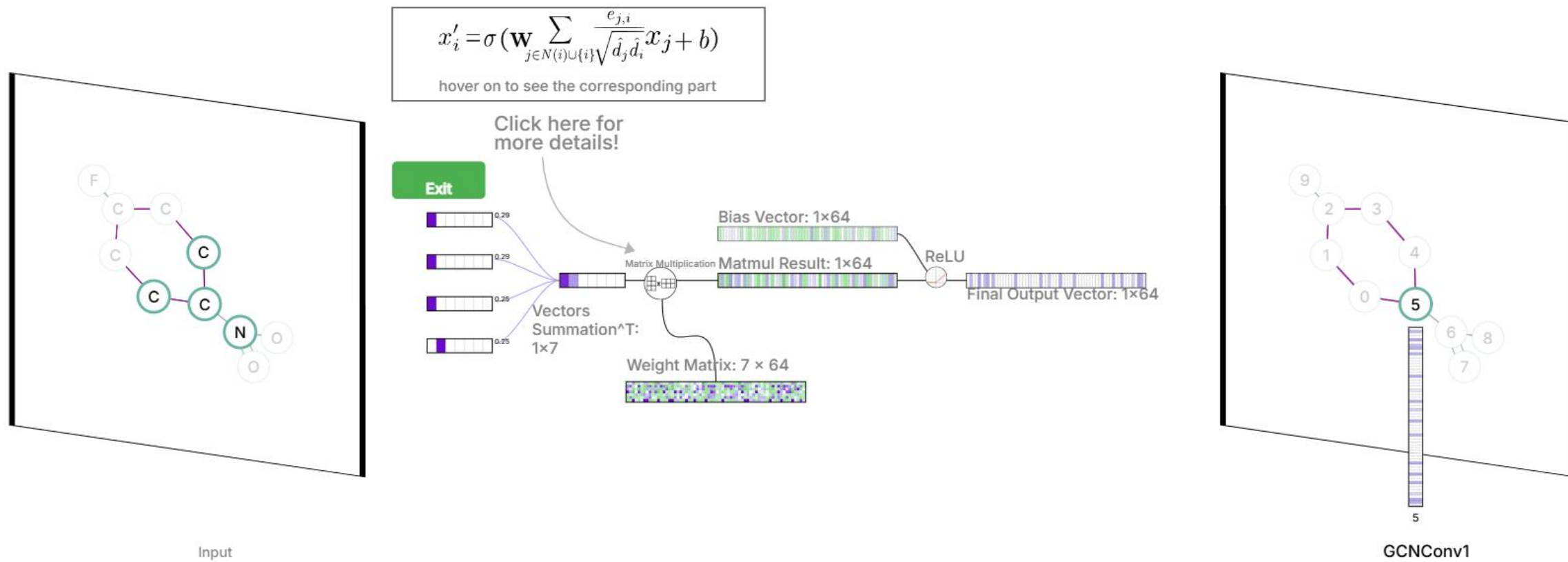
$v$ : a node in  $V$ ;  $N(v)$ : the set of neighbors of  $v$



Same vectors for node A (order 1) and node E (order 2)

# A Visual Tool for Understanding GNNs

<https://visual-intelligence-umn.github.io/GNN-101/>

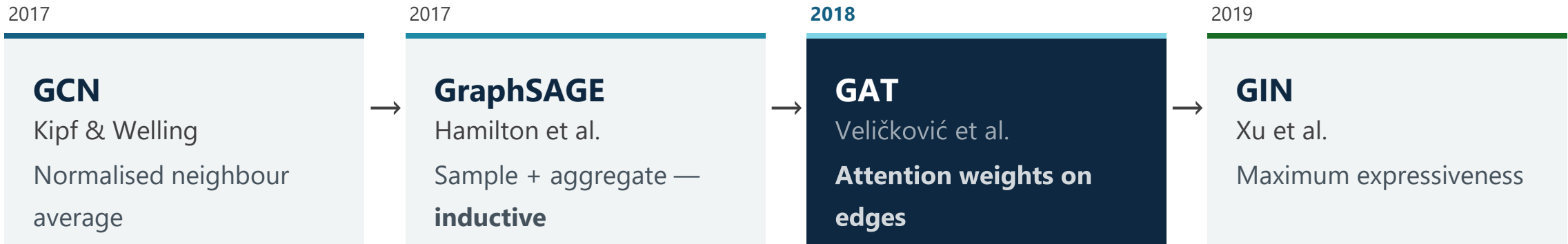


---

SECTION 3

# Base GNN Operators

# The Most Known GNN Types

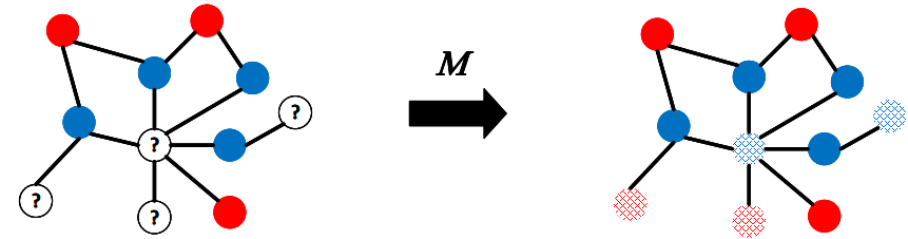


# Inductive vs Transductive

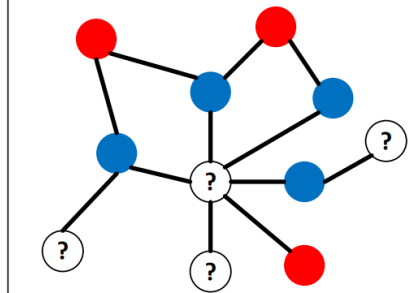
## Transductive

New unseen data samples?  
→ Must **re-train / re-compute**

Test Dataset



Training Dataset

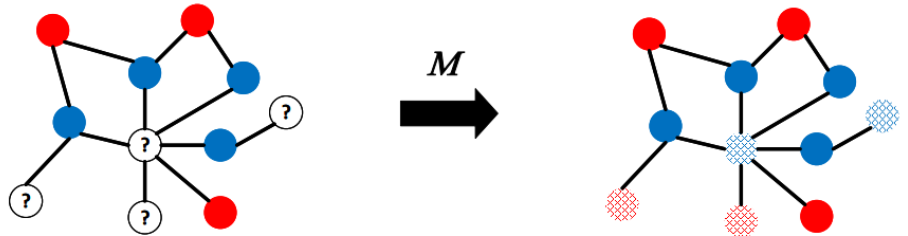


## Inductive

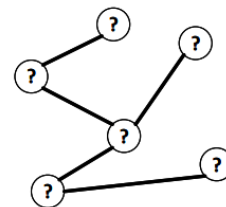
Apply out-of-the-box for new samples

**Spatial GNNs**  
(message-passing-based)

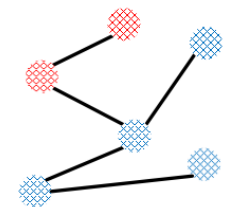
Test Dataset



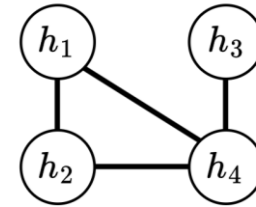
...



...



# GCN and GraphSAGE



$$h_u^{(k)} \in \mathbb{R}^H$$

Node u embedding at GNN layer (k)

## GCN

$$h_v^{(k)} = \sigma \left( W^{(k)} \sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{\hat{d}_v \hat{d}_u}} h_u^{(k-1)} \right)$$

Mean AGG including target node u, with node degrees:  $\hat{d}_v = |N(v)| + 1$

Matrix Form:

$$H^{(k)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(k-1)} W^{(k)})$$

## GraphSAGE

$$h_v^{(k)} = \sigma \left( W^{(k)} \cdot \text{CONCAT} \left( h_v^{(k-1)}, \underbrace{\frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(k-1)}}_{\text{aggregated neighbors}} \right) \right)$$

Concatenate target node

$N(v)$  often fixed in size and sampled  $\rightarrow$  scale to densely-connected graphs.

Also presents other aggregation alternatives:

Max pooling

LSTM

# Graph Isomorphism Network

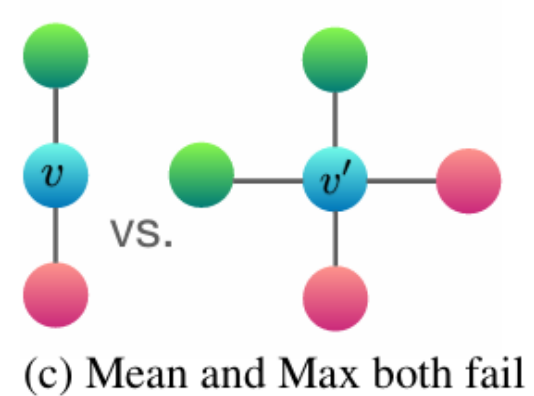
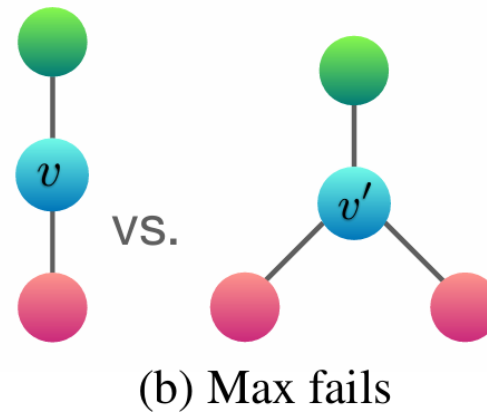
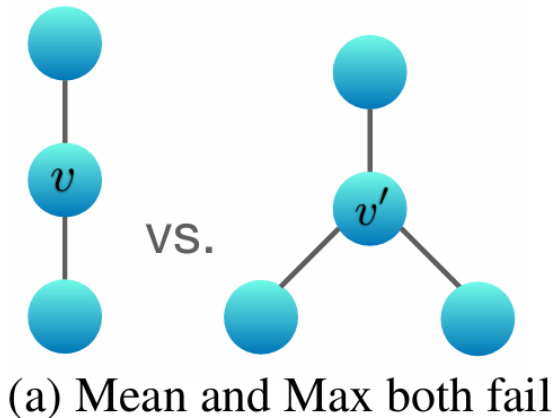
## Goal

We want aggregations that is injective over the neighbors.

$$h_v^{(k)} = \text{MLP}^{(k)} \left( (1 + \epsilon^{(k)}) h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)} \right)$$

Addition is the most injective-preserving

## Why:

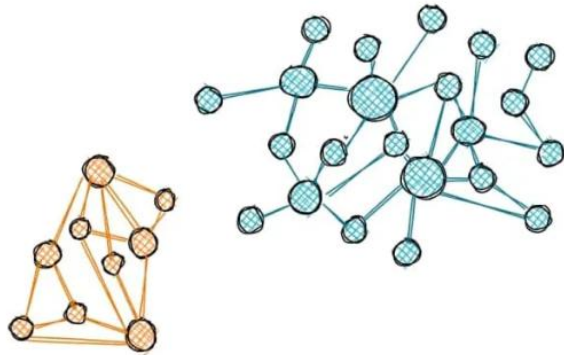


# Homophilic vs Heterophilic Graphs

## Homophilic

Nodes that are connected are expected to have similar properties.

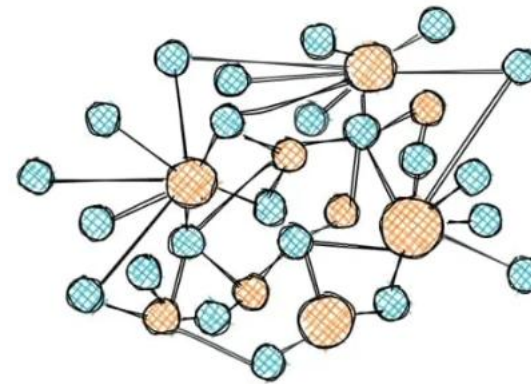
E.g., social networks, citation networks



## Heterophilic

Two neighboring nodes can have significantly different roles.

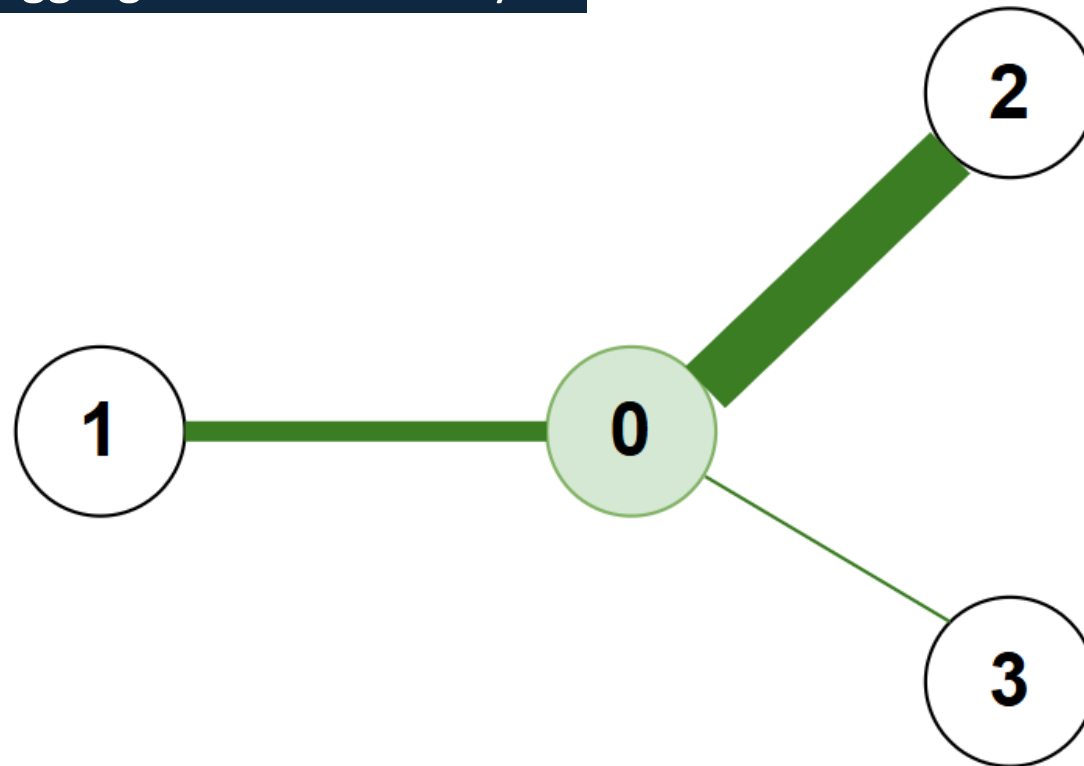
E.g., protein-protein interactions, scheduling, fraud



**We need a GNN aggregation that better distinguishes neighbors**

# Attention in a Graph

Weight Neighbor Aggregations Differently



# Graph Attention Network

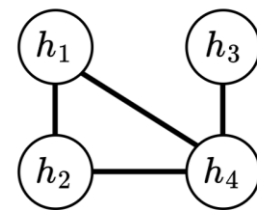
Learn to weight the neighbor contribution.

$$\vec{h}'_i = \sigma \left( \sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W} \vec{h}_j \right)$$

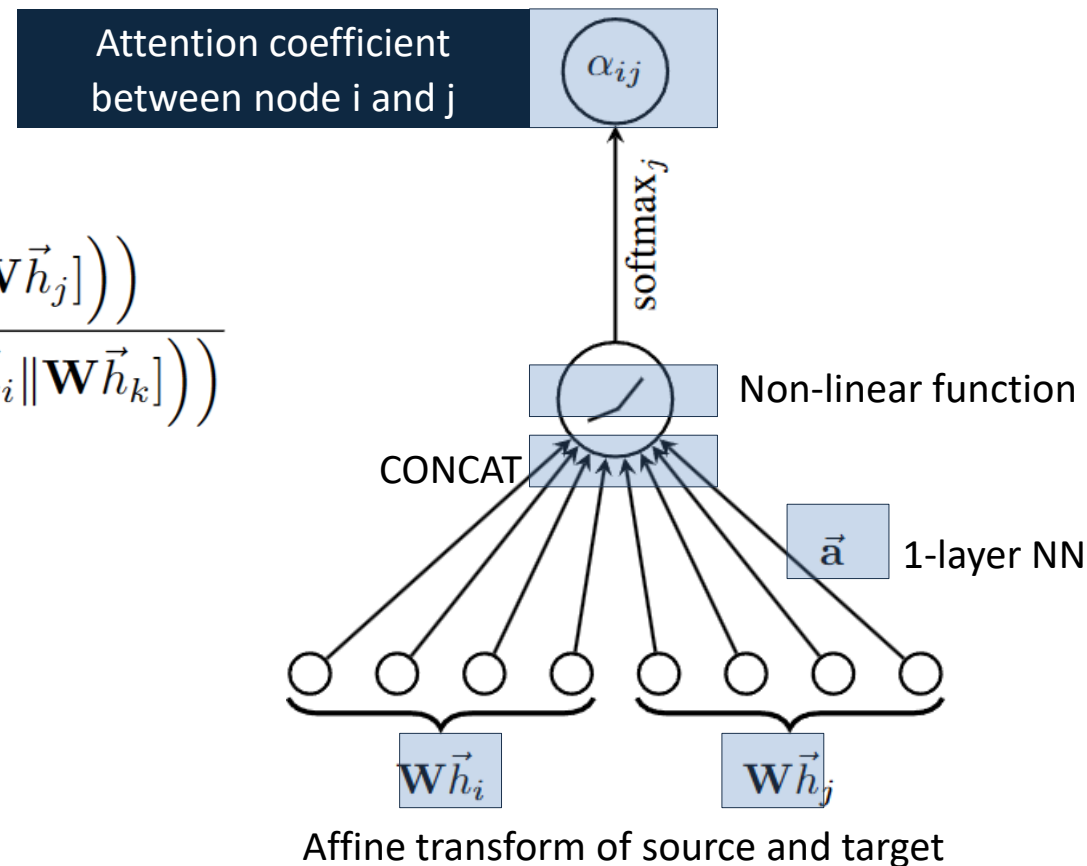
$$\alpha_{ij} = \frac{\exp \left( \text{LeakyReLU} \left( \vec{\mathbf{a}}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left( \text{LeakyReLU} \left( \vec{\mathbf{a}}^T [\mathbf{W} \vec{h}_i \parallel \mathbf{W} \vec{h}_k] \right) \right)}$$

Multi-head attention also possible

$$\vec{h}'_i = \bigparallel_{k=1}^K \sigma \left( \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right)$$

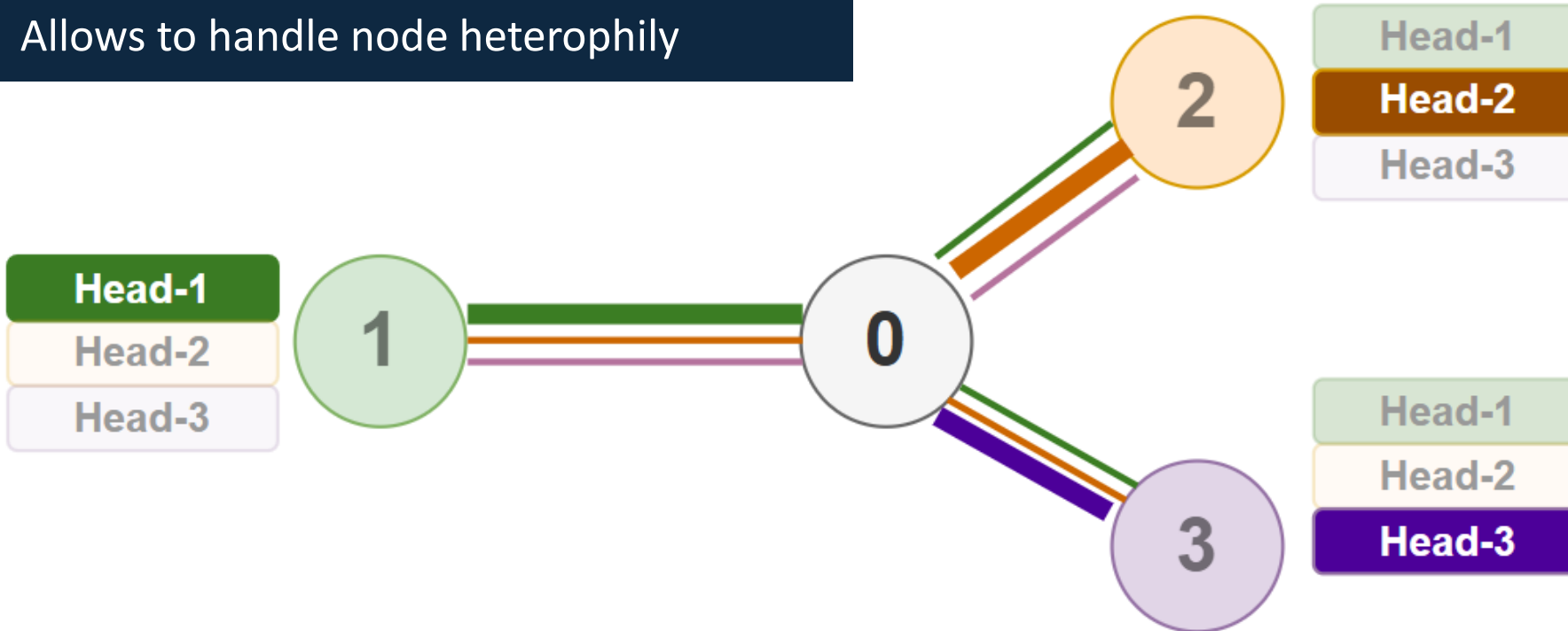


$h_u \in \mathbb{R}^H$  Node u embedding



# Multi-Head Graph Attention

Allows to handle node heterophily



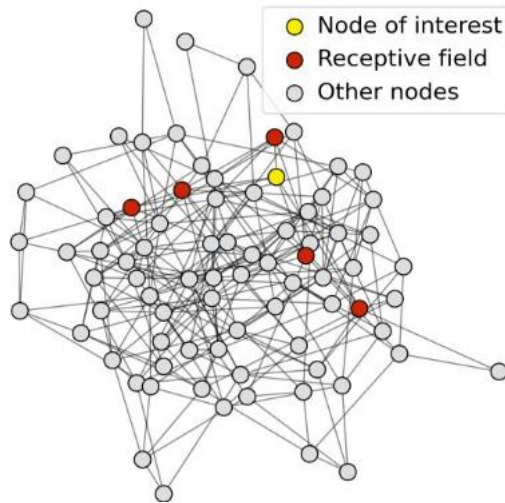
---

SECTION 4

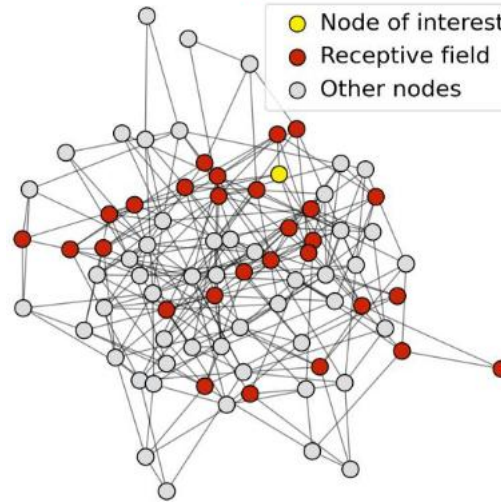
# Deep Learning for GNNs

# GNNs Receptive Field

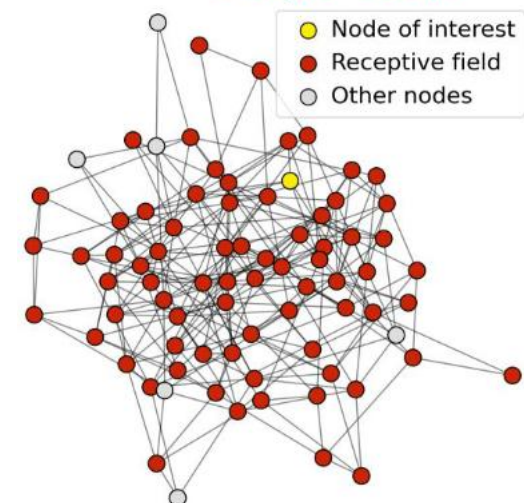
Receptive field for  
**1-layer GNN**



Receptive field for  
**2-layer GNN**

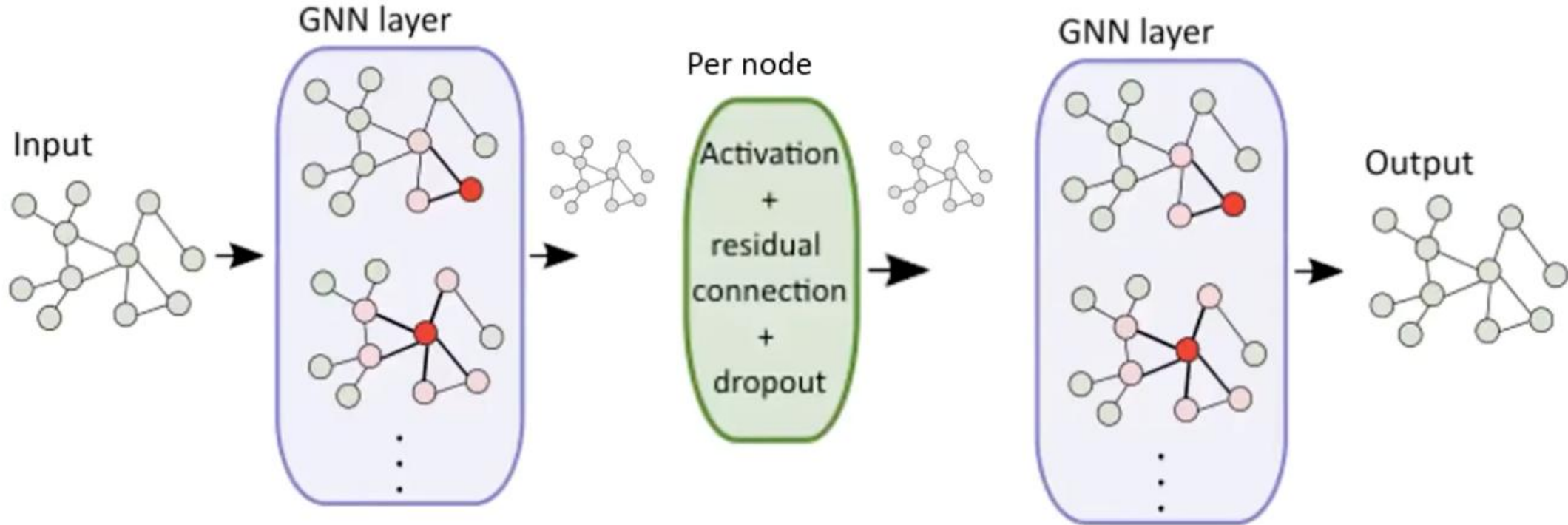


Receptive field for  
**3-layer GNN**



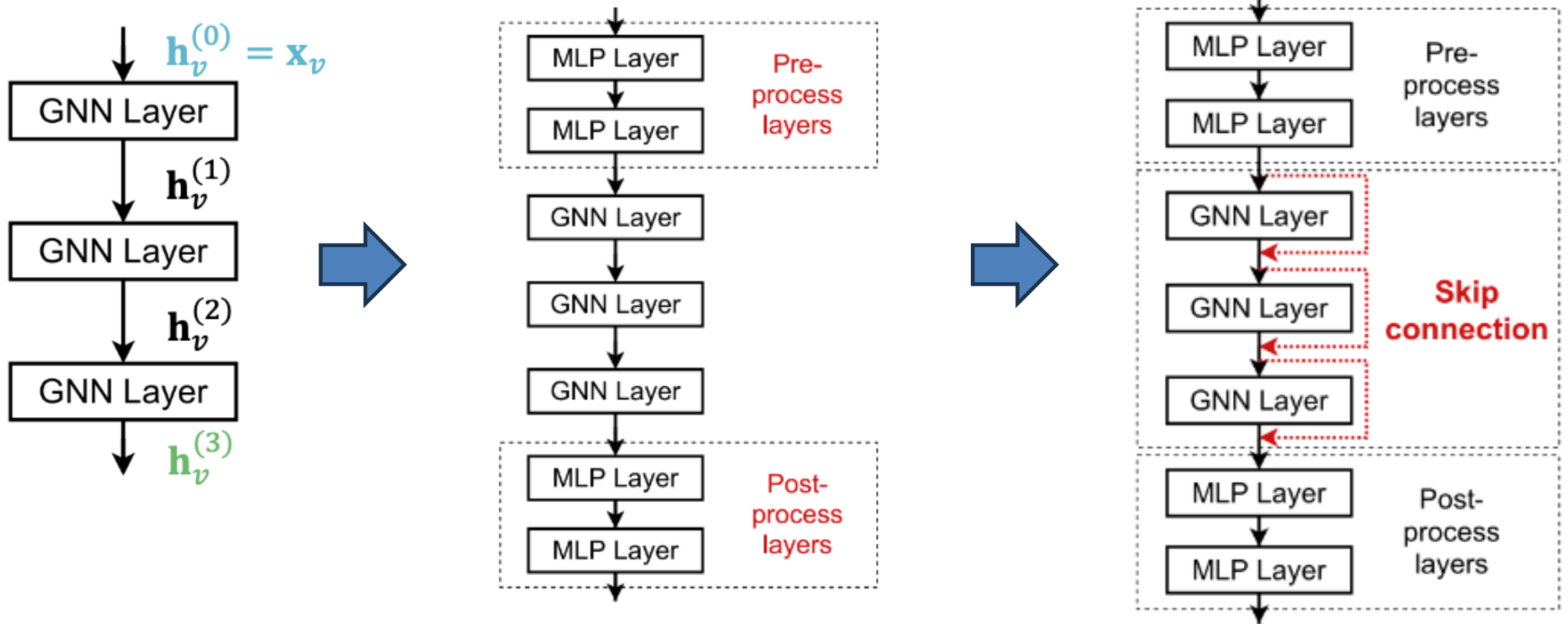
Stacking K GNN layers allows each node to gether information from its K-hop neighborhood

# Stacking GNN Layers



# Stacking GNN Layers

Include modern deep learning modules for better performance.

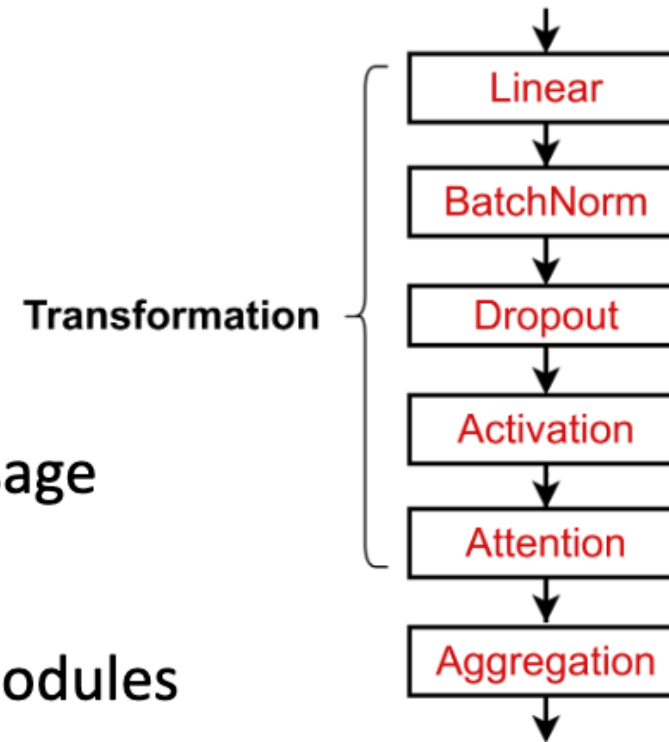


# Stacking GNN Layers

Include modern deep learning modules for better performance.

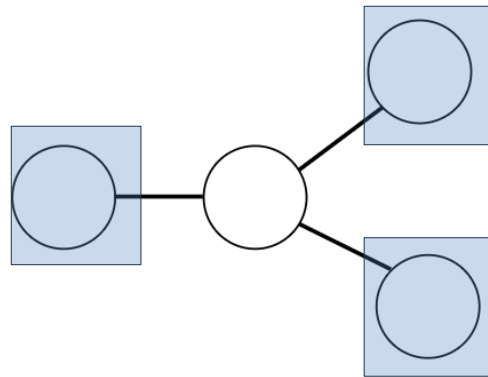
- **Batch Normalization:**
  - Stabilize neural network training
- **Dropout:**
  - Prevent overfitting
- **Attention/Gating:**
  - Control the importance of a message
- **More:**
  - Any other useful deep learning modules

## A suggested GNN Layer



# Positional Encoding in Graphs

Idea: help to distinguish better the node contributions

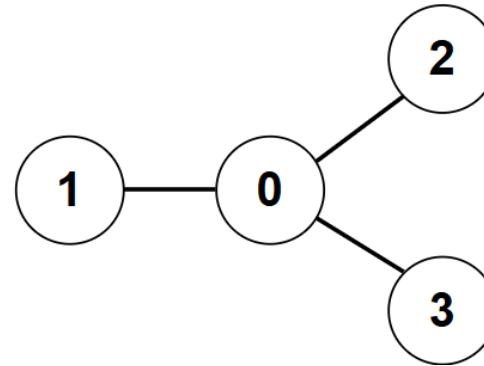


These three nodes are the same (structurally)

Input node features



Dim. 4 x D



Add another feature to X for the “position”

Input node features



Dim. 4 x (D+E)

Random Integer

Random Walks

Laplacian Eigenvectors

Degree-based Encodings

# Graph Mini-Batches for Training

One mini-batch containing multiple graphs

**IDEA:** Create one big graph of multiple independent connected components

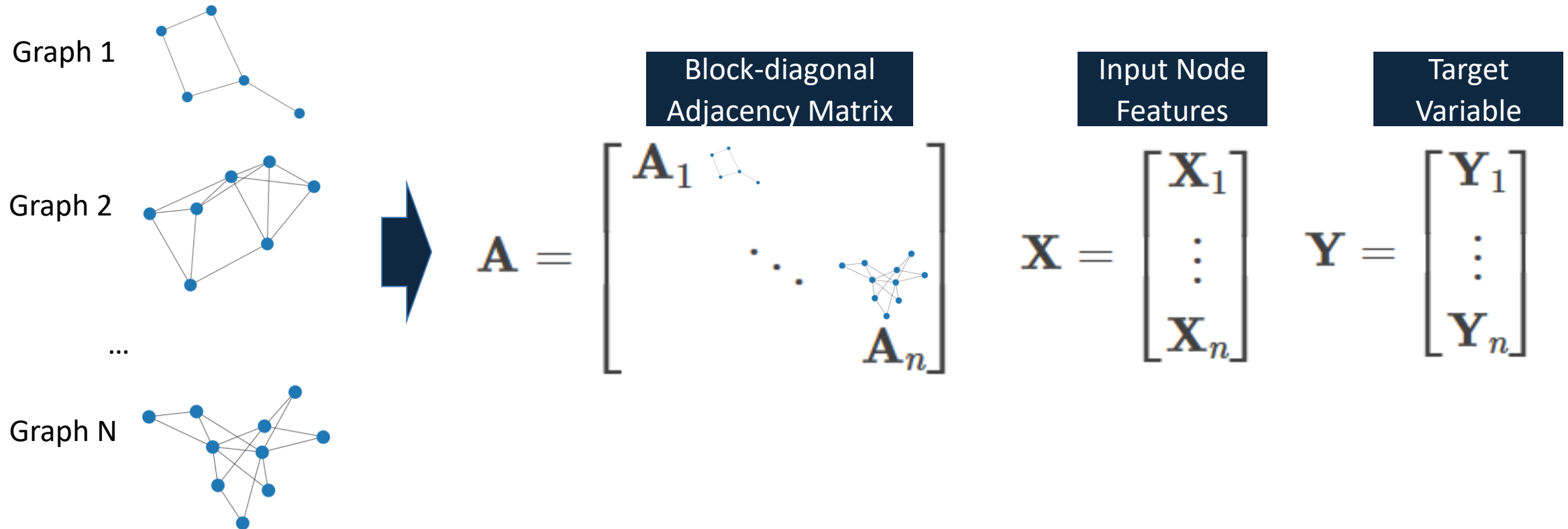


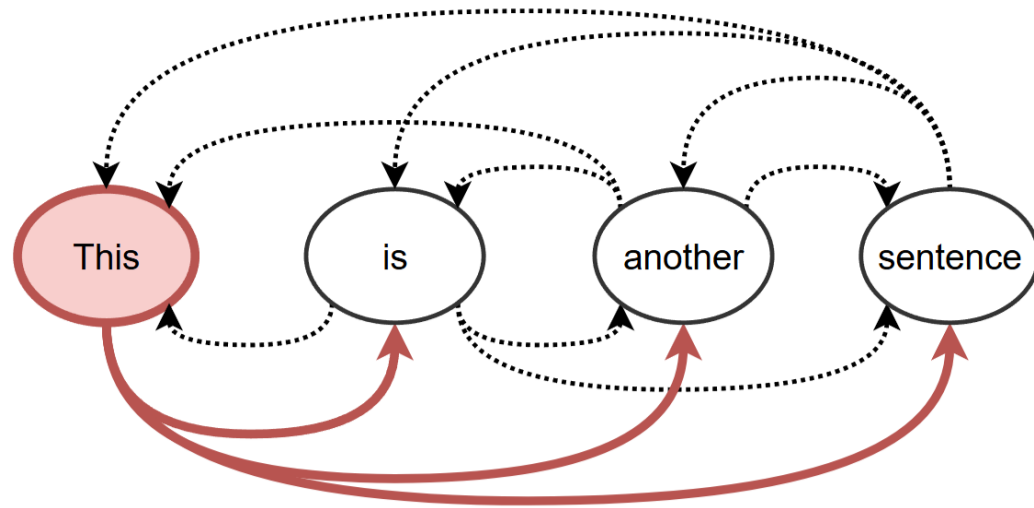
Image credit: Jure Leskovec, Stanford CS224W: Machine Learning with Graphs, <http://cs224w.stanford.edu>

---

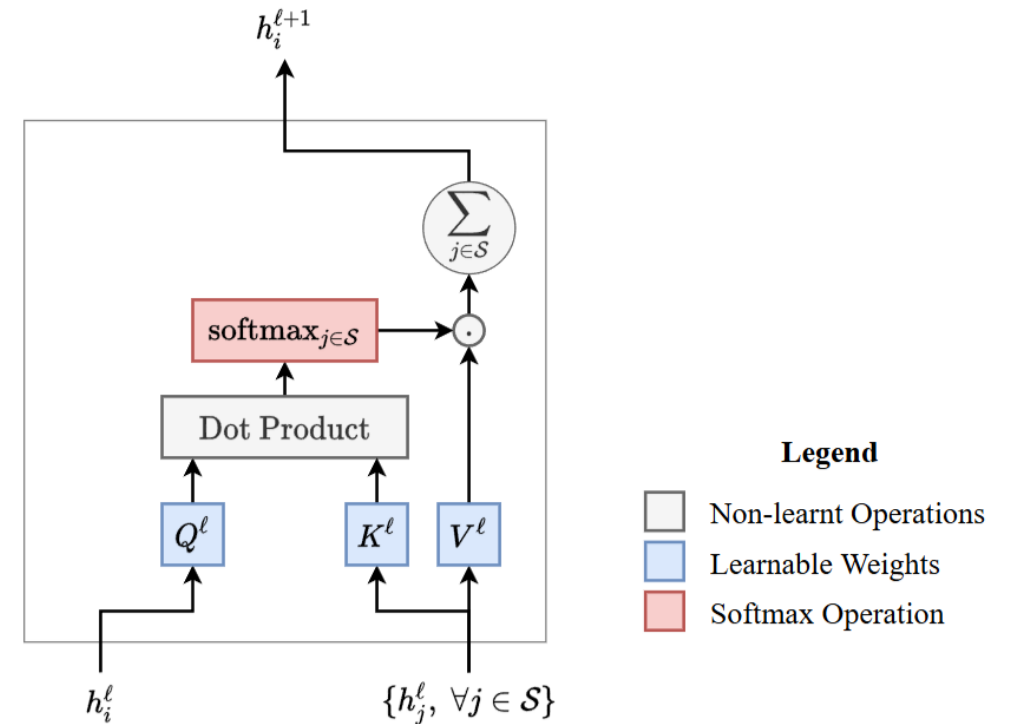
SECTION 5

# Transformers vs GNNs

## Perspective: Transformers use GNNs



Transformers process the Sentence as a Graph



Self-attention is a type of GNN



## SECTION 5

# Wrap-up

# Complementary Strengths

## LLMS EXCEL AT

Language understanding & generation

Broad world knowledge from pre-training

A flexible, prompt-based interface

## GNNS EXCEL AT

Relational pattern recognition

Propagating information through networks

Encoding features + topology jointly

# Module III → Module IV

## TWO TOOLS IN HAND

- **Module II** — a local LLM that operates over text
- **Module III** — a GNN that operates over graphs

**Open question: how do we combine them into one system?**

## WHAT'S NEXT — MODULE IV

- **Text-attributed graphs** — text on graph nodes
- **GraphRAG** — graph-based multi-hop retrieval
- **G-Retriever** — GNN subgraph retrieval → LLM

# Batch Normalization

- **Goal:** Stabilize neural networks training
- **Idea:** Given a batch of inputs (node embeddings)
  - Re-center the node embeddings into zero mean
  - Re-scale the variance into unit variance

**Input:**  $\mathbf{X} \in \mathbb{R}^{N \times D}$   
 $N$  node embeddings

**Trainable Parameters:**  
 $\gamma, \beta \in \mathbb{R}^D$

**Output:**  $\mathbf{Y} \in \mathbb{R}^{N \times D}$   
Normalized node embeddings

**Step 1:**  
**Compute the mean and variance over  $N$  embeddings**

$$\mu_j = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_{i,j}$$
$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_{i,j} - \mu_j)^2$$

**Step 2:**  
**Normalize the feature using computed mean and variance**

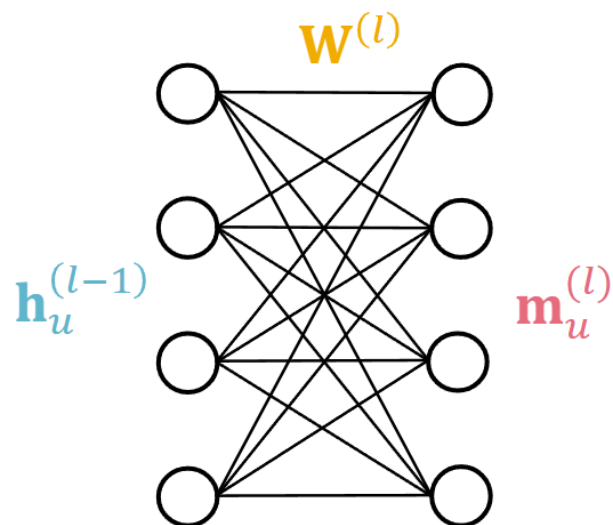
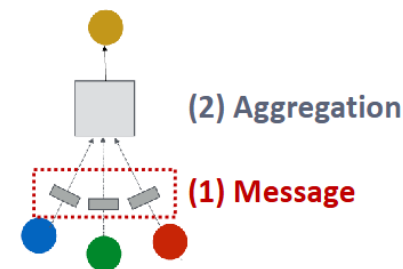
$$\hat{\mathbf{x}}_{i,j} = \frac{\mathbf{x}_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$
$$\mathbf{y}_{i,j} = \gamma_j \hat{\mathbf{x}}_{i,j} + \beta_j$$

# Dropout

- In GNN, Dropout is applied to **the linear layer in the message function**

- A simple message function with linear

layer:  $\mathbf{m}_u^{(l)} = \mathbf{W}^{(l)} \mathbf{h}_u^{(l-1)}$



Visualization of a linear layer

Dropout  
→

